

Aurora Siger — Fase 3: Operação Energética da Colônia

Relatório Técnico Integrador

Gabriel Carmona Bittencourt · Iúri Leão de Almeida · Márcio Francisco dos Santos Júnior

2026

Sumário

Aurora Siger — Fase 3: Operação Energética da Colônia	1
Nota de consolidação	2
Tabela de procedência	2
Resumo	2
1. Introdução	2
2. Organização dos dados e da colônia (item 1.1)	3
2.1 Os treze módulos — continuidade da Fase 2	3
2.2 Duas árvores N-árias sobre os mesmos dados	3
2.3 Estado sem singleton e séries temporais	3
3. Regras de decisão (item 1.2)	3
3.1 Duas camadas de decisão	3
3.2 O nível de energia como rótulo de saída	4
3.3 Déficit instantâneo $\neq$ emergência	4
4. Modelo de previsão (item 1.3)	4
4.1 Uma regressão, dois usos	4
4.2 Por que OLS, e não gradiente descendente	4
4.3 A antecipação na prática	5
5. Análise e ganho energético (item 1.4)	5
5.1 Balanço da missão	5
5.2 Controle de carga em duas camadas	5
5.3 Consumo com modelo térmico	5
5.4 Realismo: falhas e clima extremo	6
6. Arquitetura de consolidação	6
6.1 Duas branches, uma síntese	6
6.2 Dois front-ends sobre um núcleo determinístico	6
6.3 Determinismo como garantia de projeto	6
7. Conclusão	6
Referências	7

Aurora Siger — Fase 3: Operação Energética da Colônia

Atividade Integradora — Ciência da Computação (online), FIAP, 2026 Repositório: <https://github.com/iurileo-hub/FIAP-Aurora-Siger>

Nota de consolidação

A Fase 3 da atividade integradora foi desenvolvida pela equipe em um repositório próprio — <https://github.com/Gcarmonapy7/fiap-aurora-siger-fase3> — em **duas branches arquiteturalmente distintas** a partir do mesmo enunciado:

- **main** (majoritariamente Márcio): pacote `colonia_aurora/`, orientado a objetos e multi-thread, com dashboard TUI ao vivo, RNG congruencial linear (LCG) próprio, regressão por gradiente descendente e sistema de crew/eventos.
- **iuri**: pacote `colony/`, funcional/puro sobre séries temporais horárias, com clima cientificamente fundamentado (opacidade atmosférica **tau**, modelo térmico $Q = U \cdot A \cdot \Delta T$, degradação de painéis, tempestades como máquina de estados), regressão OLS de forma fechada, árvores hierárquicas e alocação de carga em quatro estágios.

Esta versão, integrada ao portfólio pessoal de Iúri Leão, é uma **consolidação** das duas branches: adota o núcleo científico da branch **iuri** como canônico e *colhe* da branch **main** o que ela tinha de melhor (dashboard, níveis de energia, LCG, falhas com auto-reparo, frente fria). O objetivo declarado foi produzir uma Fase 3 *melhor que a soma das duas*. Os três autores permanecem creditados; a curadoria e a síntese são de Iúri.

Tabela de procedência

Componente	Origem	Observação
Clima (tau, térmico, painel, tempestades FSM), OLS, hierarquias, alocação 4 estágios, 13 módulos	iuri	núcleo científico canônico
Dashboard TUI, níveis de energia CRITICAL→SURPLUS, power_factor , LCG, falhas+auto-reparo, frente fria	main	colheita, re-vestida no estilo funcional
Controle de carga em duas camadas, regressão única com dois usos, aba “Hierarquia”, adaptador SimSnapshot	consolidação	inédito em ambas as branches

Resumo

A Fase 3 simula a **operação energética** da colônia Aurora Siger — a mesma que pousou na Fase 2 — ao longo de 7 sóis marcianos (168 horas), passo a passo de uma hora. O sistema gera energia (solar, eólica, nuclear), consome-a (carga base por modo de operação somada a um termo térmico físico), armazena o saldo em bateria e **decide** continuamente como distribuir potência escassa. A contribuição conceitual da fase é a transição de um controle puramente **reativo** (cortar carga depois que falta energia) para um controle **preditivo** (antecipar a queda via regressão e agir antes), preservando o reativo como rede de segurança. Toda a aleatoriedade flui por um único gerador congruencial linear *seed-aware*, de modo que a simulação é **determinística**: a mesma seed reproduz a história hora a hora.

1. Introdução

O Aurora Siger acompanha a colônia em três atos: **decolagem** (Fase 1 — telemetria e decisão Go/No-Go), **pouso** (Fase 2 — autorização e estabilização) e agora **operação** (Fase 3 — energia e decisão contínua). A natureza do problema muda a cada ato. Nas duas primeiras fases a decisão era pontual e de curta janela. Na Fase 3 ela ganha um eixo temporal: não basta saber se há energia *agora*, é preciso estimar se haverá *adiante*.

Esse deslocamento é o fio condutor do relatório. As Seções 2 a 5 percorrem os quatro itens do enunciado — organização hierárquica (1.1), regras de decisão (1.2), previsão por regressão (1.3) e análise energética (1.4) — e a Seção 6 explica a arquitetura de consolidação que uniu as duas branches da equipe.

Todos os números citados provêm da execução canônica `run_simulation(seed=42)`, reproduzível a partir do notebook `fases/fase-3/notebook.ipynb`.

2. Organização dos dados e da colônia (item 1.1)

2.1 Os treze módulos — continuidade da Fase 2

A colônia é uma lista plana de **13 módulos**. Os módulos 1–12 preservam exatamente os nomes e prioridades dos módulos que pousaram na Fase 2 — a colônia que pousou é literalmente a que opera agora; o 13.^o (gerador eólico) foi acrescentado na Fase 3. Não se introduz uma taxonomia paralela: a continuidade narrativa é uma decisão de projeto.

Cada módulo é um `dict` simples com identidade (`id`, `name`, `type`), parâmetros físicos (`thermal_factor`, consumo por modo) e estado de runtime (`current_mode`, `broken`, `repair_hours_remaining`).

2.2 Duas árvores N-árias sobre os mesmos dados

Sobre a lista plana, `hierarchies.py` constrói **duas** árvores N-árias (classe genérica `Node`, em `tree.py`):

- **Funcional** — agrupa por função: Energy, Life Support, Command, Operations.
- **Criticidade** — agrupa por prioridade de sobrevivência: Vital → Sustenance → Expansion.

As duas árvores referenciam os **mesmos** `dicts` de módulo. Alterar `module["current_mode"]` é imediatamente visível por qualquer das árvores — não há cópia nem sincronização manual. A árvore de criticidade é a estrutura que o alocador de carga percorre: quando a oferta é escassa, ele rebaixa modos de baixo para cima (Expansão antes de Sustento, Sustento antes de Vital), e **Vital nunca desliga**.

2.3 Estado sem singleton e séries temporais

O estado da simulação é um `dict` local construído por `init_simulation(seed)` — clima, bateria, árvore de criticidade, máquinas de estado de tempestade e frente fria, e o RNG. **Não há singleton global** (decisão herdada da branch `iuri`, contra o `DataStorage` da `main`): duas simulações podem coexistir no mesmo processo sem interferência. A cada passo, 16 séries temporais são anexadas ao `history` (geração por fonte, consumo, bateria, clima, nível de energia, slope, contagem de quebrados, alertas), e é sobre essas séries que toda a análise opera.

3. Regras de decisão (item 1.2)

3.1 Duas camadas de decisão

O sistema decide em dois níveis, deliberadamente separados:

1. **Camada didática** (`decision.py`) — `evaluate_rules(snapshot)` é uma função pura que recebe um retrato do estado (`energy_kw`, `consumption_kw`, `storm`) e devolve uma lista de ações legíveis (“ALERTA: reduzir consumo”, “ATIVAR MODO ECONOMIA”, “EMERGÊNCIA ENERGÉTICA”, “ALERTA CLIMÁTICO”). É a camada inspecionável exigida pelo enunciado, no formato exato pedido pelo currículo.
2. **Camada estrutural** (`allocation.py`) — o *load shedding* em quatro estágios, descrito na Seção 5.2, que efetivamente rebaixa os modos dos módulos.

3.2 O nível de energia como rótulo de saída

A máquina `CRITICAL → LOW → NOMINAL → HIGH → SURPLUS` (`energy_levels.py`) **não controla** os módulos: ela é o **rótulo de saída**, computado de `bateria% + slope` OLS. O fluxo é unidirecional — física → nível → apresentação/decisão. O ponto preditivo está aqui: um `slope` suficientemente negativo **rebaixa** o nível antes de a bateria efetivamente cair (detalhado na Seção 4.2).

Na execução canônica, a distribuição de níveis ao longo das 168 horas foi:

Nível	Horas
CRITICAL	14
LOW	45
NOMINAL	41
HIGH	54
SURPLUS	14

3.3 Déficit instantâneo ≠ emergência

Um resultado revelador: em **93 das 168 horas** a geração instantânea ficou abaixo do consumo (`status = risk` em `analyze_balance`), mas **apenas 7 horas** dispararam alerta real de emergência. A diferença é a **bateria**: à noite o solar zera e só o nuclear sustenta a base, então o consumo instantâneo supera a geração — mas a bateria absorve o vale e recarrega ao longo do dia. É a tradução numérica do argumento central da fase: o “agora” (déficit instantâneo) assusta; o estado real (bateria mais tendência) tranquiliza. Decidir só pelo instantâneo seria reagir a um falso alarme 93 vezes; decidir pela tendência é o passo preditivo.

4. Modelo de previsão (item 1.3)

4.1 Uma regressão, dois usos

`prediction.py` implementa a regressão linear por mínimos quadrados **à mão**, na forma fechada exigida pelo enunciado (sem `numpy/sklearn`):

$$a = \frac{\sum (x - \bar{x})(y - \bar{y})}{\sum (x - \bar{x})^2}, \quad b = \bar{y} - a\bar{x}$$

A mesma função `linear_regression(xs, ys)` serve a **dois** propósitos:

1. **Previsão eólica** (`fit_wind_power_model`): treina sobre os pares (`wind_ms`, `wind_generation_kw`) acima do *cut-in*, prevendo a potência eólica a partir do vento. Na execução canônica o modelo ajustado foi **energia ≈ 2,50·vento − 7,50** (kW).
2. **Slope preditivo** (`fit_energy_trend`): treina sobre a janela recente dos *deltas* de energia (geração menos consumo) e devolve (`slope`, `predicted_next_delta`) — a inclinação da tendência e a projeção de um passo à frente.

4.2 Por que OLS, e não gradiente descendente

A branch `main` usava gradiente descendente para a tendência; a consolidação o substituiu pela forma fechada. A OLS fechada é **exata**, não tem taxa de aprendizado para calibrar, não diverge e dispensa o *clamp* anti-exploração que o gradiente exigia. Para janelas curtas (poucas dezenas de pontos), é mais barata, mais previsível e — num sistema que se quer auditável — mais fácil de explicar.

4.3 A antecipação na prática

Em 21 das 168 horas o slope esteve abaixo de $-0,5$ kW/h (queda relevante). Nessas janelas, `energy_level()` rebaixou o rótulo um degrau *antes* de a porcentagem de bateria cruzar o limiar correspondente — exatamente a preempção que distingue o preditivo do reativo. A previsão informa a decisão; não a substitui (ver Seção 7).

5. Análise e ganho energético (item 1.4)

5.1 Balanço da missão

`analysis.py` agrega o `history` em métricas de balanço. Na execução canônica:

Métrica	Valor
Geração média	86,5 kW
Consumo médio	85,2 kW
Geração máxima	145,2 kW
Bateria final	328,1 / 500,0 kWh (65,6 %)
Horas com tempestade	63
Horas com alerta de emergência	7

O *breakdown* por fonte mostra o nuclear como base estável e o solar/eólico como contribuição variável:

Fonte	Potência média	Participação
Nuclear	71,0 kW	82,0 %
Solar	8,3 kW	9,6 %
Eólica	7,2 kW	8,4 %

Os momentos críticos da missão: **pior déficit** no sol 3, hora 04 ($-71,1$ kW, durante tempestade leve, com solar zerado de madrugada); **maior excedente** no sol 6, hora 12 ($+56,5$ kW, céu limpo ao meio-dia).

5.2 Controle de carga em duas camadas

A síntese central da consolidação combina as filosofias opostas das duas branches em **defesa em profundidade**, sem dupla-contagem:

1. **Camada descentralizada** — **power_factor (da main)**: conforme a bateria cai, estrangula *suavemente* o alvo de consumo de cada módulo, numa escala contínua de 1,0 a 0,2. É preventiva e graduada — começa a economizar cedo, quando ainda é barato.
2. **Camada centralizada** — **load shedding 4 estágios (da iuri)**: percorre a árvore de criticidade e, quando a oferta ainda não cobre a demanda já atenuada, rebaixa modos de baixo para cima. É a rede de segurança estrutural.

A composição evita dupla-contagem por ordem de operação: (a) computa-se o **power_factor** da bateria; (b) ele escala os **alvos adequate/surplus**; (c) a alocação decide os **modos** contra a oferta usando esses alvos já escalados. Verificou-se numericamente que o consumo real após a alocação cabe sob a oferta sem contagem duplicada.

5.3 Consumo com modelo térmico

`current_consumption_kw(module, climate, power_factor)` é puro: `base_por_modulo × power_factor + termo_térmico`. O termo térmico segue o modelo físico de perda por envelope $Q = U \cdot A \cdot \Delta T$ escalado

pelo `thermal_factor` do módulo, com um ganho interno passivo que cobre a perda até cerca de $-86,7\text{ }^{\circ}\text{C}$ (abaixo disso o aquecimento elétrico entra). O termo é somado **mesmo com o módulo “off”** — habitats pressurizados não podem congelar — e zera naturalmente quando `thermal_factor == 0`.

5.4 Realismo: falhas e clima extremo

- **Falhas com auto-reparo (`failures.py`):** cada módulo ativo tem $\sim 0,5\text{ } \%/ \text{hora}$ de chance de falhar (via LCG). Ao falhar, sai da geração e do consumo; o reparo é automático e temporizado (sem sistema de crew). Na execução canônica houve até **3 módulos quebrados simultaneamente**, com pelo menos um em reparo em 108 das 168 horas.
- **Frente fria (`climate.py`):** além das tempestades de poeira (máquina de estados), uma frente fria derruba a temperatura por janelas de horas, disparando pico de consumo térmico. A temperatura mínima registrada foi $-97,7\text{ }^{\circ}\text{C}$, com 38 horas abaixo do limiar de $-86\text{ }^{\circ}\text{C}$ que aciona o aquecimento elétrico.

6. Arquitetura de consolidação

6.1 Duas branches, uma síntese

O desafio não foi escrever a Fase 3 do zero, mas **unir** duas implementações maduras e filosoficamente opostas. A decisão de brainstorming foi a abordagem “núcleo `iuri` canônico + colheita da `main`” (ver tabela de procedência). Ficaram **fora de escopo** (YAGNI), por conflitarem com as convenções do portfólio ou com a continuidade da Fase 2: o sistema de crew (substituído por reparo automático), o *spawn* dinâmico de módulos (quebraria os 13 módulos), o gradiente descendente (substituído por OLS), o singleton `DataStorage` (contra “sem globais”) e o tick de relógio de parede (substituído por horas).

6.2 Dois front-ends sobre um núcleo determinístico

O núcleo de simulação é exposto por dois front-ends que **espelham** o padrão notebook + `mgpeb` da Fase 2:

- **Notebook** (`fases/fase-3/notebook.ipynb`) — narrativa headless via `run_simulation()`, com gráficos. Importa toda a lógica de `aurora_siger.operations` (nunca inline).
- **CLI aurora** (`cli.py` + `dashboard.py`) — dashboard TUI ao vivo de **6 abas** (Visão Geral, Energia, Sensores, Módulos, Eventos e **Hierarquia** — a árvore de criticidade visualizada, item 1.1 ao vivo). A thread existe apenas para o *padding* visual; o conteúdo permanece determinístico dada a seed.

O dashboard lê os dados por um adaptador fino, `SimSnapshot`, que expõe `.get()/ .history()/ .modules()` sobre o estado do simulador — substituindo o singleton `DataStorage` da branch `main` sem reescrever as primitivas de renderização ANSI colhidas dela.

6.3 Determinismo como garantia de projeto

Toda fonte de aleatoriedade — clima e falhas — passa pelo **mesmo** LCG injetado em `state["rng"]`. Combinado com o reset de estado de runtime dos módulos a cada nova simulação, isso garante que duas execuções com a mesma seed produzam históricos **bit-a-bit idênticos** — verificado por diff de dois runs no modo headless do CLI.

7. Conclusão

A Fase 3 fecha o arco decolagem → pouso → operação e materializa o objetivo do enunciado: evoluir de um sistema reativo para um preditivo. A lição de engenharia, porém, é que essa evolução **não substitui** o reativo — o estratifica. O *load shedding* e o `power_factor` permanecem como fundação; sobre eles, uma regressão de poucos coeficientes vigia a tendência e permite agir com folga, antes do limiar.

A previsão é uma hipótese, não uma certeza: uma OLS linear sobre janela curta extrapola mal sob regime não-estacionário (tempestade, frente fria), e por isso o humano-no-loop continua necessário — eco direto da reflexão ética da Fase 1. Um sistema que só reage sobrevive a cada hora; um que também prevê começa a ter chance de planejar o sol seguinte. A reflexão completa está no ensaio `docs/fase-3/reactivo-a-preditivo.md`.

Referências

- Repositório do portfólio (esta consolidação): <https://github.com/iurileo-hub/FIAP-Aurora-Siger>
- Repositório original da equipe (branches `main` e `iuri`): <https://github.com/Gcarmonapy7/fiap-aurora-siger-fase3>
- Especificação de design da consolidação: `docs/superpowers/specs/2026-05-27-fase-3-operations-consolidacao-`
- Ensaio reflexivo: `docs/fase-3/reactivo-a-preditivo.md`